

Guía de Actividades Práctico-Experimentales Nro. 011

1. Datos Generales

Asignatura	Diseño Digital
Ciclo	5to
Unidad	Dos
Resultado de aprendizaje de la unidad	R2. Desarrolla proyectos embebidos, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad
Práctica Nro.	11
Título de la Práctica	FreeRTOS: comunicación entre tareas (colas) y sincronización.
Nombre del Docente	José O. Guamán Q.
Fecha	16-05-2026 18-05-2026
Horario	Martes 12h30 – 13h30 Jueves 11h30 – 13h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Comprender y aplicar los principios de planificación de tareas en sistemas embebidos tipo FreeRTOS, utilizando prioridades, colas (colas) y sincronización, mediante el diseño de un sistema multitarea basado en un caso real de ingeniería.

- Implementar el sistema embebido multitarea basado en un contexto real de aplicación.
- Implementar comunicación entre tareas mediante colas (colas) para intercambio de datos.
- Analizar el comportamiento del sistema bajo diferentes prioridades y condiciones de sincronización.

3. Materiales y reactivos:

1x Placa Arduino Uno R3.

3x LEDs (Sugerencia: 1 Rojo, 1 Amarillo, 1

Verde). 3x Resistencias de 220 Ohm (o 330

Ohm).

Cables de conexión (Jumpers) para el breadboard.

4. Equipos y herramientas

Resultados de la practica 10

5. Procedimiento / Metodología

A. Fase 1

1. Definición de Requisitos

- **Funcionalidad**

Nº Requisito	Descripción
RF-01	El sistema debe simular la apertura y cierre de una ventana mediante un servomotor, dependiendo del nivel de luz detectado por el sensor LDR.
RF-02	El sistema debe detectar movimiento mediante un sensor PIR y notificar visualmente al usuario encendiendo un LED.
RF-03	El sistema debe medir la temperatura ambiental mediante un sensor LM35.
RF-04	El sistema debe activar una alarma sonora mediante un buzzer cuando la temperatura se encuentre fuera del rango permitido.
RF-05	El sistema debe mostrar en una pantalla LCD los valores de temperatura, luz, movimiento y estado de la ventana.
RF-06	El sistema debe enviar información al monitor serial para permitir la supervisión del comportamiento del sistema.

- **Restricciones de tiempo**

El sistema presenta las siguientes restricciones de tiempo:

- El sistema nunca debe detenerse o apagarse durante su funcionamiento.
- El sistema debe leer los datos de los sensores cada 1 segundo.
- El sistema debe procesar los datos de luz, movimiento y temperatura de forma continua mediante tareas independientes.
- El sistema debe actualizar la pantalla LCD cada vez que recibe nuevos datos procesados.
- El sistema debe controlar la apertura o cierre del servo cuando exista un cambio en el nivel de luz.
- El sistema debe activar el buzzer cuando la temperatura sea menor o igual a 19.9 °C o mayor o igual a 31.0 °C.
- El sistema debe utilizar FreeRTOS para organizar las tareas de adquisición, procesamiento y actuación.

- **Costos**

Cantidad	Elemento Electrónico	Precio	Total
2	Arduino UNO	12.00	12.00
1	Sensor LDR	0.50	0.50
1	Sensor PIR	3.00	3.00
1	Sensor LM35	3.25	3.25

1	Servomotor	3.5	3.5
1	Buzzer	0.45	0.45
1	Pantalla LCD I2C 16x2	4.00	4.00
1	LED	0.10	0.10
1	Protoboard y cables	2.00	2.00
		Total	28.80

2. Modelado del sistema

• Diagramas

A continuación, se presenta un diagrama inicial de componentes en la protoboard, el cual permite visualizar la conexión entre el Arduino UNO, los sensores de entrada, los actuadores de salida y la pantalla LCD.

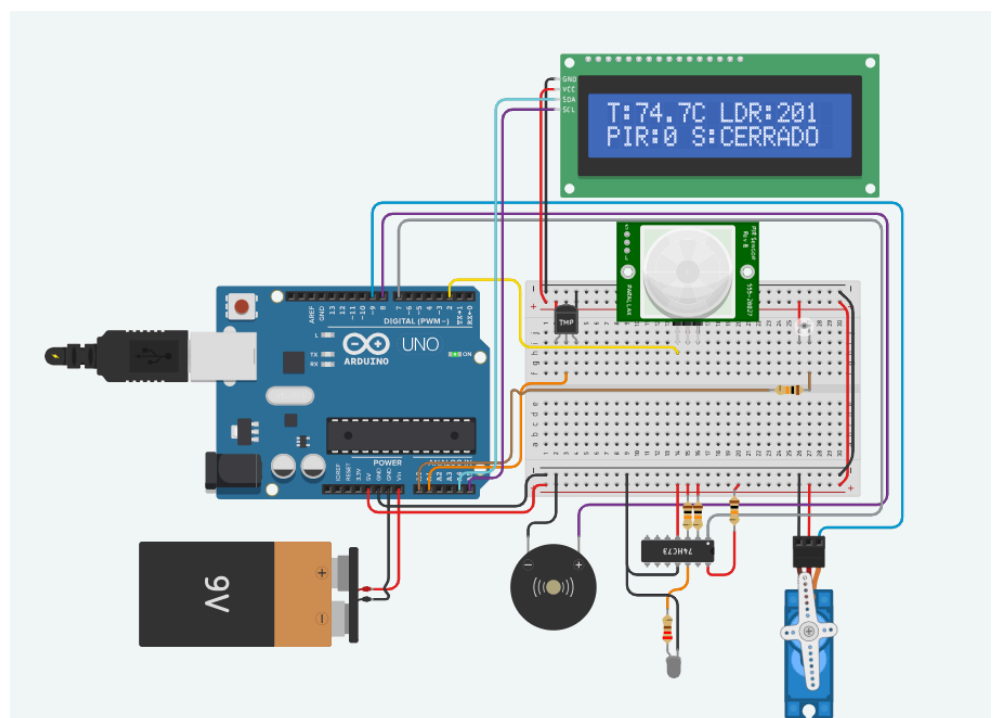


Figura 1. Diagrama de componentes

• Modelos de comportamiento

En esta práctica, el comportamiento del sistema está definido mediante reglas específicas:

- El sensor LDR mide la cantidad de luz recibida.
- Si el valor del LDR es mayor a 500, el sistema simula la apertura de una ventana moviendo el servo a 180 grados.
- Si el valor del LDR es menor o igual a 500, el sistema simula el cierre de la ventana moviendo el servo a 0 grados.
- El sensor PIR detecta movimiento.
- Si el PIR detecta movimiento, se activa un LED conectado al pin 7.
- El sensor LM35 mide la temperatura ambiental.
- Si la temperatura es menor o igual a 19.9 °C o mayor o igual a 31.0 °C, se activa el buzzer como alarma.
- La pantalla LCD muestra la temperatura, el valor del LDR, el estado del PIR y el estado del servo.
- El monitor serial muestra los valores de los sensores para supervisar el sistema.

Cada una de estas reglas representa una respuesta automática del sistema ante

cambios en el ambiente, permitiendo controlar sensores y actuadores de manera coordinada.

- **Máquinas de estados finitos**

Para este sistema se pueden definir los siguientes estados:

- **Estado Normal:**

La temperatura se encuentra dentro del rango permitido. El buzzer permanece apagado. El sistema continúa leyendo sensores y mostrando información en la LCD.

- **Estado de Alarma por Temperatura:**

La temperatura es menor o igual a 19.9 °C o mayor o igual a 31.0 °C. El buzzer se activa para alertar al usuario.

- **Estado con Movimiento Detectado:**

El sensor PIR detecta movimiento. El LED conectado al pin 7 se enciende como señal visual.

- **Estado Ventana Abierta:**

El valor del LDR es mayor a 500. El servo se posiciona en 180 grados, simulando la apertura de la ventana.

- **Estado Ventana Cerrada:**

El valor del LDR es menor o igual a 500. El servo se posiciona en 0 grados, simulando el cierre de la ventana.

- **Definir funcionalidad**

El sistema se modela como un proceso de lectura, procesamiento y actuación. Primero, los sensores LDR, PIR y LM35 capturan información del entorno. Luego, estos datos son enviados a una cola de FreeRTOS para que otra tarea los procese. Según los valores recibidos, el sistema decide si debe abrir o cerrar la ventana mediante el servo, encender el LED por detección de movimiento o activar el buzzer por temperatura fuera del rango establecido.

Finalmente, una tarea de actuación se encarga de ejecutar las salidas físicas y mostrar la información en la pantalla LCD y en el monitor serial.

B. Fase 2: Diseño de Hardware y Software

1. Hardware

- **Arquitectura**

En esta práctica se utilizó un microcontrolador ATmega328P integrado en el Arduino UNO. Este microcontrolador permite leer sensores analógicos y digitales, controlar actuadores y comunicarse con periféricos externos.

El sistema está compuesto por sensores de entrada, actuadores de salida y una pantalla LCD I2C. Los sensores utilizados son el LDR, el PIR y el LM35. Como actuadores se utilizan un servomotor, un buzzer y un LED. La pantalla LCD permite mostrar los datos principales del sistema en tiempo real.

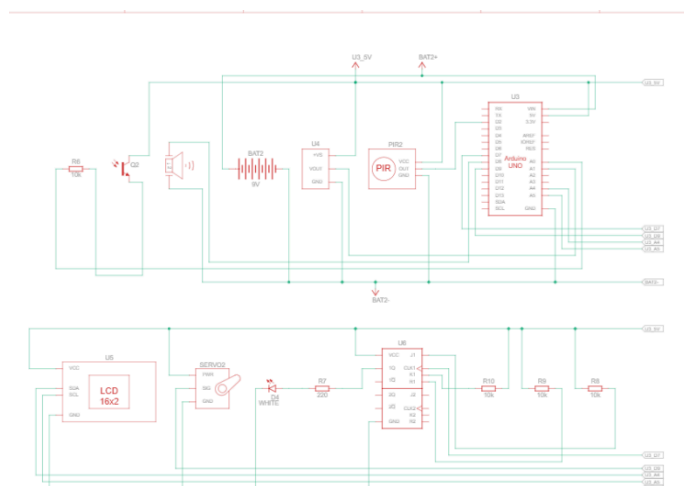


Figura 2. Diagrama Esquemático

- **Memoria**

El Arduino UNO cuenta con memoria Flash, SRAM y EEPROM. En este sistema, la memoria Flash almacena el programa compilado, mientras que la SRAM se usa para almacenar variables temporales, estructuras de datos, colas de FreeRTOS y valores capturados por los sensores.

- **Periféricos de entrada/salida**

En esta práctica se utilizaron diversos periféricos de entrada y salida. Como entradas se emplearon un sensor LDR conectado al pin A0 para medir la intensidad de luz, un sensor LM35 conectado al pin A1 para medir la temperatura y un sensor PIR conectado al pin digital 2 para detectar movimiento. Como salidas se utilizaron un servomotor conectado al pin digital 9 para simular la apertura y cierre de una ventana, un buzzer conectado al pin digital 8 para generar alertas sonoras, un LED conectado al pin digital 7 para indicar la detección de movimiento, una pantalla LCD I2C conectada mediante comunicación I2C para mostrar información del sistema y el monitor serial para visualizar los valores capturados y supervisar el funcionamiento general del proyecto.

2. Software

- **Lenguajes**

El programa se desarrolla en Arduino IDE utilizando C/C++. Este lenguaje permite controlar directamente los pines del microcontrolador, leer sensores, activar actuadores y gestionar estructuras de datos.

- **RTOS**

En esta práctica se utiliza FreeRTOS para implementar multitarea en Arduino UNO. El sistema está dividido en tres tareas principales.

La tarea TaskAdquisicion se encarga de leer los sensores LDR, PIR y LM35 cada 1 segundo. Posteriormente, envía los datos obtenidos a una cola llamada queueSensores, permitiendo que otras tareas puedan utilizarlos sin interferir con el proceso de adquisición.

La tarea TaskProcesamiento recibe los datos desde la cola de sensores y evalúa las condiciones del sistema. Con base en los valores obtenidos, determina las acciones que deben ejecutarse, como abrir o cerrar el servo, activar el LED o encender el buzzer.

La tarea TaskActuacion recibe las acciones procesadas y controla físicamente los actuadores del sistema. Además, se encarga de actualizar la pantalla LCD y enviar información al monitor serial para facilitar la supervisión del funcionamiento del sistema.

El uso de FreeRTOS permite organizar mejor los procesos del programa, evitando que todo dependa de una única función loop(). Gracias a esta estructura, cada tarea puede ejecutarse de manera independiente y coordinada, mejorando la organización y el comportamiento general del sistema.

- **Eficiencia**

La eficiencia del sistema se logra mediante la separación de responsabilidades en tareas independientes. Cada tarea cumple una función específica, lo que facilita la lectura, procesamiento y actuación del sistema.

Además, se utiliza vTaskDelay() en lugar de delay() para permitir que FreeRTOS administre el tiempo entre tareas. Esto evita bloqueos innecesarios y permite que el sistema continúe ejecutando otras tareas mientras una de ellas se encuentra en espera.

También se optimiza el control del servomotor, ya que solo se envían pulsos cuando existe un cambio en el estado del servo. Esto evita repetir movimientos innecesarios y reduce el consumo de procesamiento.

- **Modularidad**

La modularidad se observa en la división del programa en tres tareas principales: adquisición, procesamiento y actuación. Esta organización permite separar claramente la lectura de sensores, la toma de decisiones y el control de salidas.

Además, el uso de estructuras como SensorData y ActionData permite organizar

los datos de forma ordenada. La comunicación entre tareas se realiza mediante colas, lo cual mejora la claridad del programa y permite que cada tarea trabaje de forma independiente.

3. Cocreación

- **Concepto Crítico**

Durante el desarrollo del sistema se integraron conocimientos de programación, electrónica básica, sensores, actuadores, comunicación I2C y sistemas operativos en tiempo real. El sistema desarrollado combina sensores de luz, movimiento y temperatura con salidas físicas como el servomotor, el buzzer, el LED y la pantalla LCD.

- **Optimización de Rendimiento y energía**

La optimización del rendimiento se consigue mediante el uso de tareas independientes y retardos controlados con `vTaskDelay()`. Esto permite que el procesador no quede bloqueado innecesariamente y que las tareas se ejecuten de manera ordenada.

C. Fase 3: Implementación e integración

1. De código a Bits

- **Compilación**

Esta etapa se refiere al proceso mediante el cual el código escrito en C/C++ se traduce a instrucciones que puede entender el microcontrolador ATmega328P. Para este proceso se utiliza Arduino IDE, que permite verificar errores de sintaxis, compilar el programa y cargarlo en la placa Arduino UNO.

- **Lenguaje de máquina**

El lenguaje de máquina está formado por instrucciones binarias que son interpretadas directamente por la CPU del microcontrolador. Una vez compilado el programa, las tareas de FreeRTOS, la lectura de sensores, el control del servo, la activación del buzzer, el encendido del LED y la comunicación con la pantalla LCD se convierten en instrucciones ejecutables para el Arduino UNO.

- **Flujos de bits para hardware programable**

El programa compilado genera un archivo binario que contiene todas las instrucciones necesarias para implementar el sistema. Este flujo de bits se carga en la memoria Flash del microcontrolador y permite que el Arduino ejecute las tareas de adquisición, procesamiento y actuación.

2. Integración de plataformas

- **Memoria**

La memoria del ATmega328P almacena tanto el programa como los datos necesarios para su ejecución. La memoria Flash contiene el código compilado, mientras que la SRAM almacena variables temporales, estructuras de datos, colas y pilas de las tareas de FreeRTOS.

- **Periféricos**

En la práctica y en puntos anteriores se han mencionados los periféricos de entrada y salida.

- **CPU en PCB**

La CPU se encuentra integrada dentro del microcontrolador ATmega328P, el cual está montado en la placa Arduino UNO. Esta integración permite conectar directamente sensores, actuadores y módulos externos mediante los pines de entrada y salida.

3. Grabado

- **Memoria flash del microcontrolador**

Una vez compilado el programa, el archivo binario se transfiere a la memoria Flash del ATmega328P mediante el puerto USB del Arduino UNO. La memoria Flash es no volátil, por lo que conserva el programa incluso después de desconectar la alimentación.

D. Fase 4

1. Simulación vs pruebas en la placa

- **Entorno virtual**

A continuación, se presenta el entorno de simulación utilizado para comprobar el funcionamiento del sistema antes de implementarlo físicamente. En este caso, se puede emplear Tinkercad o una simulación equivalente para verificar la lectura de sensores y el comportamiento de los actuadores.

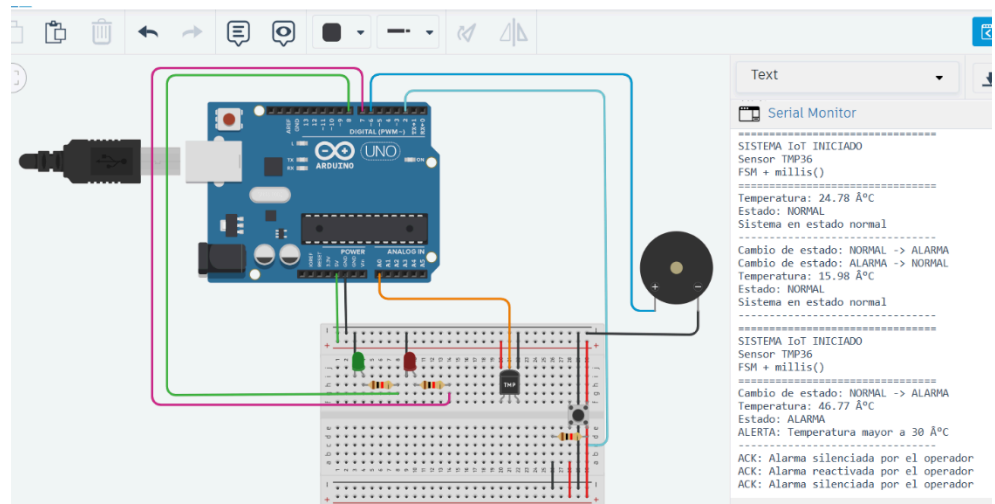


Figura 3. Entorno de simulación

- **Hardware real**

En esta sección se presentan las imágenes correspondientes del sistema implementado en físico. En el hardware real se comprueba la conexión del sensor LDR, sensor PIR, sensor LM35, servomotor, buzzer, LED y pantalla LCD I2C.

También se verifica que los datos mostrados en la pantalla LCD coincidan con los valores enviados al monitor serial.

- **Tiempos y logística**

El sistema responde correctamente a las restricciones de tiempo planteadas al inicio de la metodología. La tarea de adquisición realiza lecturas cada 1 segundo, mientras que las tareas de procesamiento y actuación trabajan de acuerdo con los datos recibidos mediante las colas.

Gracias al uso de FreeRTOS, el sistema mantiene una ejecución organizada y permite separar la lectura de sensores, el procesamiento de datos y el control de actuadores.

2. Aceptación de usuario

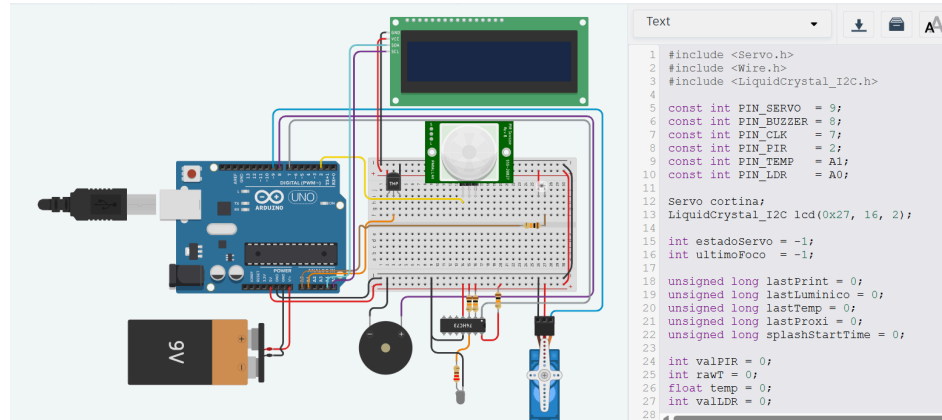
- **Cumplimiento de especificaciones**

Al finalizar esta práctica y luego de pasar por los procesos de especificación de requisitos, simulación, control de errores e implementación en un sistema real, se puede concluir que el sistema cumple con las especificaciones planteadas.

El sistema permite leer datos de luz, movimiento y temperatura, procesarlos mediante tareas de FreeRTOS y generar respuestas físicas mediante el servomotor, el buzzer, el LED y la pantalla LCD. Además, la comunicación por monitor serial permite supervisar el comportamiento del sistema en tiempo real.

6. Resultados esperados:

- Sistema embebido multitarea funcional implementado y operando en un contexto de aplicación real.



```

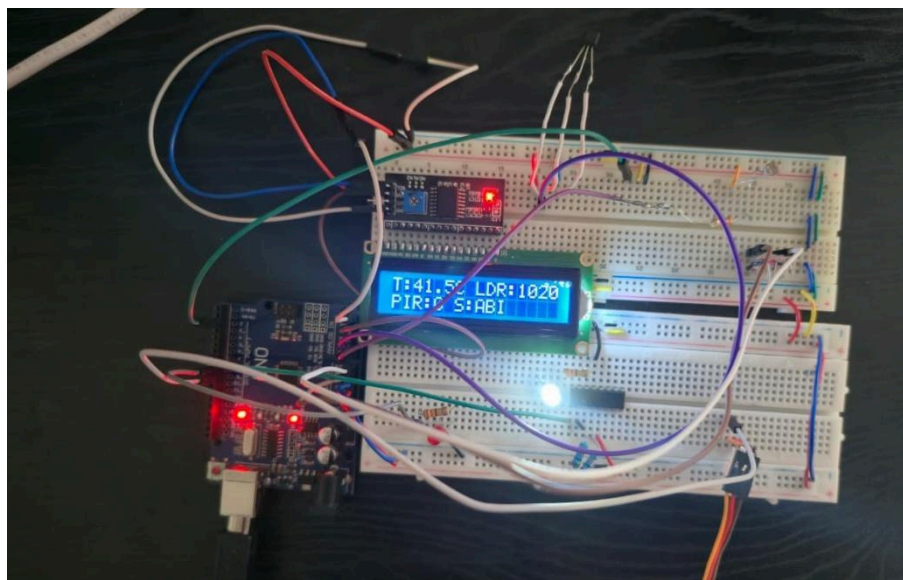
void TaskProcesamiento(void *pvParameters) {
    (void) pvParameters;
    SensorData receivedData;
    ActionData action;

    for (;;) {
        if (xQueueReceive(queueSensores, &receivedData, portMAX_DELAY) == pdPASS) {
            // Lógica de decisiones
            action.servoAngle = (receivedData.ldrValue > 500) ? 180 : 0;
            action.ledActive = (receivedData.pirValue == HIGH);
            action.alarmActive = (receivedData.tempValue <= 19.9 || receivedData.tempValue >= 31.0);

            // Traspaso de datos para la pantalla
            action.tempValue = receivedData.tempValue;
            action.ldrValue = receivedData.ldrValue;
            action.pirValue = receivedData.pirValue;

            xQueueSend(queueAcciones, &action, portMAX_DELAY);
        }
    }
}

```



7. Preguntas de Control:

- **¿Qué es un sistema operativo en tiempo real (OS) y cuál es su función en un sistema embebido?**

Es un sistema operativo diseñado para garantizar que diferentes tareas se ejecuten dentro de límites de tiempo específicos. En un sistema embebido permite administrar recursos como procesador, memoria y periféricos de entrada y salida para que el sistema responda adecuadamente incluso en situaciones críticas.

- **¿Cuál es la diferencia entre un sistema secuencial y un sistema multitarea?**

La diferencia principal es la forma en la que se ejecutan las tareas. Un sistema secuencial ejecuta las tareas una por una por lo que dos tareas no pueden hacer uso de un recurso al mismo tiempo y eso puede provocar condiciones de carrera. Por otra parte, un sistema multitarea puede ejecutar varias tareas a la vez mediante mecanismos de comunicación y colas.

- **¿Qué se entiende por tarea (task) en un sistema multitarea?**

Una tarea es una unidad independiente de ejecución dentro de un sistema multitarea. Cada tarea tiene su propio conjunto de instrucciones, variables y estado de ejecución.

- **¿Cómo influye la prioridad de una tarea en su ejecución dentro del sistema?**

Es de suma importancia ya que garantiza el orden de ejecución de las tareas y permite que las tareas de mayor prioridad se ejecuten antes. De este modo, el sistema puede responder de forma efectiva a diversos escenarios.

- **¿Qué es una cola (queue) y cuál es su propósito en la comunicación entre tareas?**

Una cola es una estructura de datos tipo FIFO, donde el primer dato que entra es el primero que sale. Su propósito es permitir el intercambio seguro y ordenado de información entre tareas.

- **¿Cuáles son las ventajas de utilizar colas frente a variables globales para el intercambio de información?**

Las colas ofrecen varias ventajas para el intercambio de información ya que evitan el conflicto de acceso simultáneo, mantienen el orden de los mensajes, reducen el riesgo de corrupción de datos y facilitan el diseño modular del software.

- **¿Qué ocurre cuando una cola se encuentra llena o vacía?**

Esto dependerá de la configuración de la cola. Por ejemplo, en caso de cola llena no se pueden insertar datos nuevos y la tarea podría bloquearse o descartar información. Asimismo, en caso de cola vacía no existen los datos disponibles para leer y la tarea receptora puede quedar esperando hasta que llegue nueva información.

- **¿Qué mecanismos de sincronización pueden utilizarse para coordinar tareas en un RTOS?**

Para evitar que dos tareas produzcan interbloqueos, existen varios mecanismos, entre ellos semáforos, mutexes, colas, eventos o notificaciones de tareas. Cada uno tiene diferentes programaciones y son útiles en determinadas áreas.

- **¿Qué es una condición de carrera y cómo puede evitarse?**

Una condición de carrera ocurre cuando dos o más tareas acceden simultánea a un recurso compartido y el resultado depende del orden de ejecución. Para evitarlas se utilizan mecanismos de sincronización como semáforos, mutexes, colas, eventos o notificaciones de tareas.

- **¿Qué es la inversión de prioridades y qué impacto puede tener en el desempeño del sistema?**

La inversión de prioridades ocurre cuando una tarea de baja prioridad bloquea un recurso que necesita una tarea de alta prioridad. Esto puede provocar comportamientos inesperados del sistema como retrasos impredecibles, incumplimiento de restricciones temporales y pérdida de rendimiento y confiabilidad.



8. Evaluación

9. Criterio	Excelente	Bueno	Bajo
Montaje y funcionamiento (100%)	Funciona completo (3)	Parcial (1)	No funciona (0)
Funcionamiento virtual (100%)	Funciona completo (2)	Parcial (1)	No funciona (0)
Documento (objetivos resueltos, tener su propia metodología, documentar bien los resultados, preguntas de control con referencias, conclusiones)	Todos los puntos (2)	Parcial (1)	No valido (0)
Defensa (Todos deben intervenir, y responder preguntas. Es por individual)	3	1	0

10. Bibliografía

- Semana 11. Quezada, Jasiel. 2026



11. Elaboración y Aprobación

Elaborado por	José Guamán Docente	 Validar únicamente en FirmaEC. Firmado electrónicamente por: JOSE OSWALDO GUAMAN QUINCHE
Aprobado por	Edison L Coronel Romero Director de Carrera	